

试题一 (15 分)

阅读下列说明和图，回答问题 1 至 4，将答案填入答题纸的对应栏内。

【说明】

某公司拟开发一个共享单车系统，采用北斗定位系统进行单车定位，提供针对用户的 APP 以及微信小程序，基于 Web 的管理与监控系统。该共享单车系统的主要功能如下。

1) 用户注册登录。用户在 APP 段端输入手机号并获取验证码后进行注册，将用户信息进行存储。用户登录后显示用户所在位置周围的单车。

2) 使用单车。

①扫码/手动开锁。通过扫描二维码或手动输入编码获取开锁密码，系统发送开锁指令进行开锁，系统修改单车状态，新建单车行程。

②骑行单车。单车定时上传位置，更新行程。

③锁车结账。用户停止使用或手动锁车并结束行程后，系统根据已设置好的计费规则及使用时间自动结算，更新本次骑行的费用并显示给用户，用户确认支付后，记录行程的支付状态，系统还将重置单车的开锁密码和单车状态。

3) 辅助管理。

①查询。用户可以查看行程列表和行程详细信息。

②保修。用户上报所在位置或单车位置以及单车故障信息并进行记录。

4) 管理与监控

①单车管理及计费规则设置。商家对单车基础信息，状态等进行管理，对计费规则进行设置并存储。

②单车监控。对单车，故障，行程等进行查询统计。

③用户管理。管理用户信用与状态信息，对用户进行查询统计。

现采用结构化方法对共享单车系统进行分析与设计，获得如图 1-1 所示的上下文数据流程图和图 1-2 所示的 0 层数据流图。

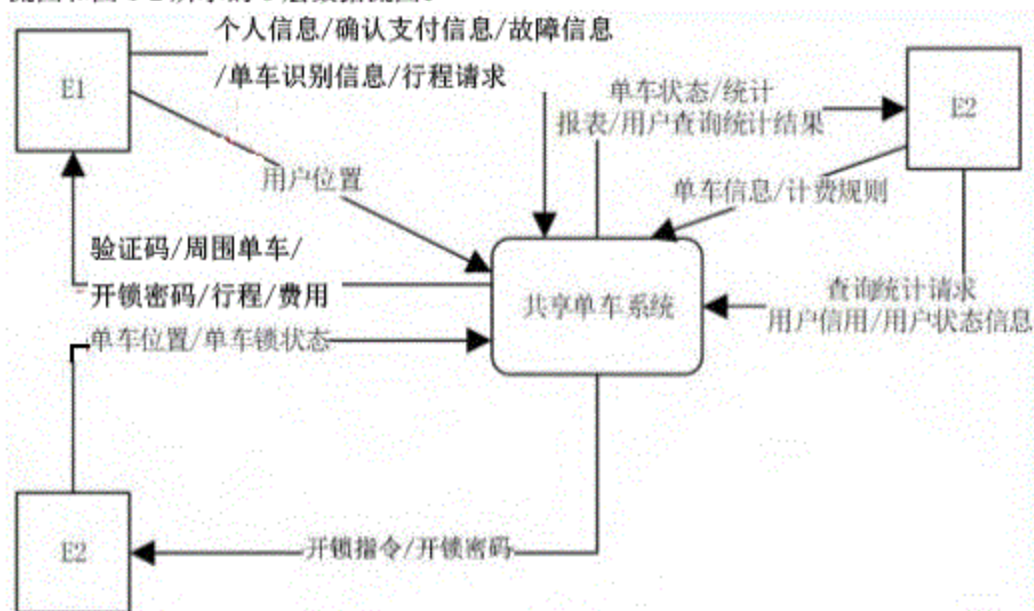


图 1-1 上下文数据流图

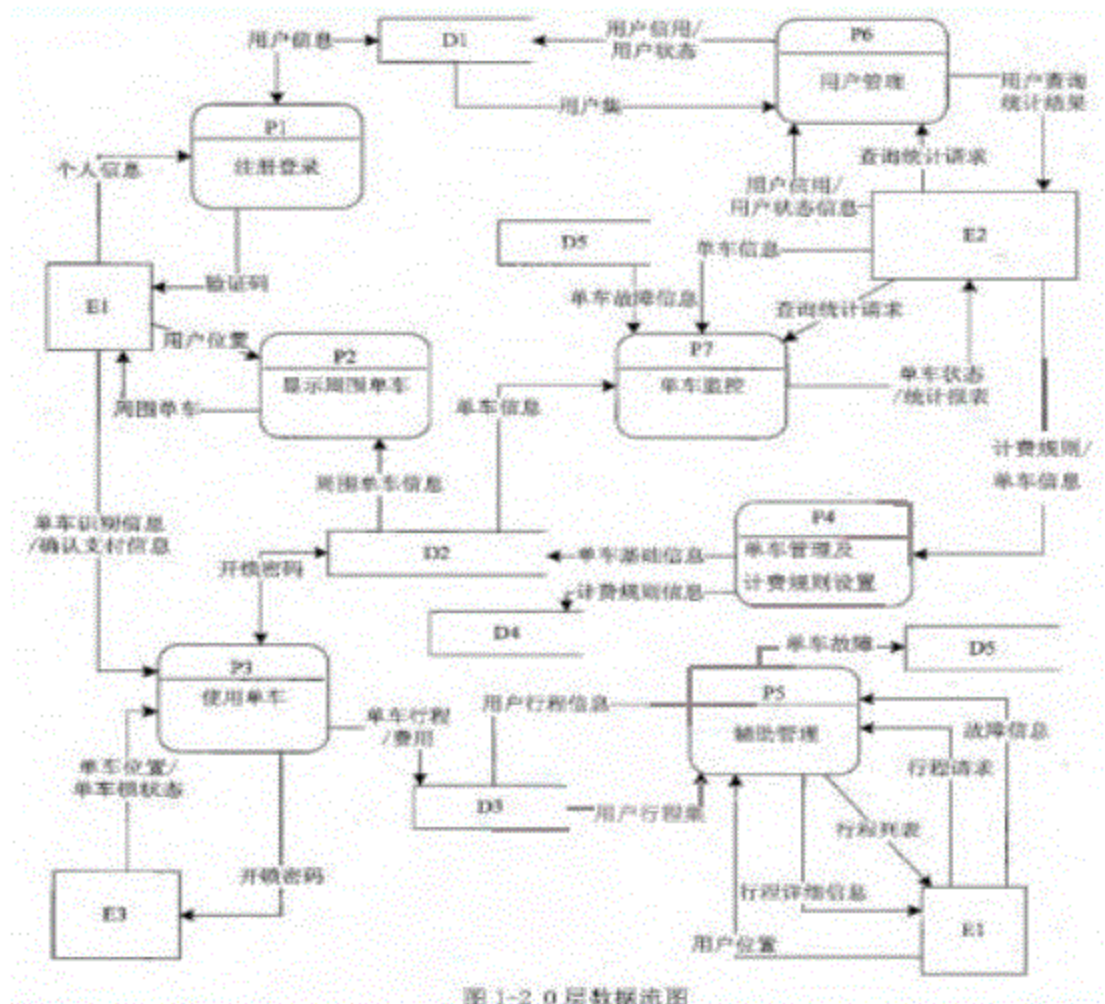


图 1-2 0 层数据流图

【问题 1】(3 分)

使用说明中的词语，给出图 1-1 中的实体 E1~E3 的名称。

【问题 2】(5 分)

使用说明中的词语，给出图 1-2 中的数据存储 D1~D5 的名称。

【问题 3】(5 分)

根据说明和图中术语及符号，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】(2 分)

根据说明中术语，说明“使用单车”可以分解为那些子加工？

试题二 (共 15 分)

阅读下列说明，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

M 公司为了便于开展和管理各项业务活动，提高公司的知名度和影响力，拟构建一个基于网络的会议策划系统。

【需求分析结果】

该系统的部分功能及初步需求分析的结果如下：

(1) M 公司旗下有业务部，策划部和其它部门。部门信息包括部门号，部门名，主管，联系电话和邮箱号。每个部门只有一名主管，只负责本部门的工作，且主管参照员工关系的员工号；一个部门有多名员工，每个员工属于且仅属于一个部门。

(2) 员工信息包括员工号, 姓名, 职位, 联系方式和薪资。职位包括主管, 业务员, 策划员等。业务员负责受理用户申请, 设置受理标志。一名业务员可以受理多个用户申请, 但一个用户申请只能由一个业务员受理。

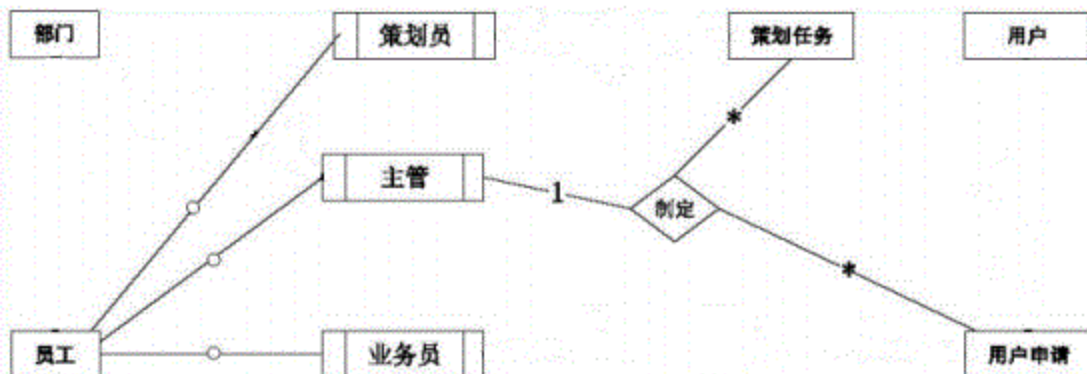
(3) 用户信息包括用户号, 用户名, 银行账号, 电话, 联系地址。用户号唯一标识用户信息中的每一个元组。

(4) 用户申请信息包括申请号, 用户号, 会议日期, 天数, 参会人数, 地点, 预算费用和受理标志。申请号唯一标识用户申请信息中的每一个元组, 且一个用户可以提交多个申请, 但一个用户申请只对应一个用户号。

(5) 策划部主管为已受理的用户申请制定会议策划任务。策划任务包括申请号, 任务明细和要求完成时间。申请号唯一标识策划任务的每一个元组。一个策划任务只对应一个已受理的用户申请, 但一个策划任务可由多名策划员参与执行, 且一名策划员可以参与执行多项策划任务。

【概念模型设计】

根据需求阶段收集的信息, 设计的实体联系图(不完整)如图 2-1 所示。



【关系模式设计】

部门 (部门号, 部门名, 部门主管, 联系电话, 邮箱号)

员工 (员工号, 姓名, (a), 联系方式, 薪资)

用户 (用户名, (b), 电话, 联系地址)

用户申请 (申请号, 用户号, 会议日期, 天数, 参会人数, 地点, 受理标志, (c))

策划任务 (申请号, 任务明显, (d))

执行 (申请号, 策划员, 实际完成时间, 用户评价)

【问题 1】(5 分)

根据问题描述, 补充五个联系, 完成图 2-1 的实体联系图, 联系名可用联系 1, 联系 2, 联系 3, 联系 4 和联系 5 表示, 联系类型为 1:1, 1:n 和 m:n (或 1:1, 1:* 和 *:*)

【问题 2】(4 分)

根据题意, 将关系模式中的空 (a) ~ (d) 补充完整, 并填入答题纸的位置上。

【问题 3】(4 分)

给出“用户申请”和“策划任务”关系模式的主键和外键。

【问题 4】(2 分)

请问“执行”关系模式的主键为全码的说法正确吗? 为什么?

试题三 (共 15 分)

阅读下列说明, 回答问题 1 至问题 3, 将解答填入答题纸的对应栏内。

【说明】

某大学拟开发一个用于管理学术出版物 (Publication) 的数字图书馆系统, 用户可以从该系统查询或下载已发表的学术出版物。系统的主要功能如下:

1. 登录系统。系统的用户 (User) 仅限于该大学的学生 (Student), 教师 (Faculty) 和其它工作人员 (Staff)。在访问系统之前, 用户必须使用其校园账号和密码登录系统。

2. 查询某位作者 (Author) 的所有出版物。系统中保存了会议文章 (ConfPaper), 期刊文章 (JournalArticle) 和校内技术报告 (TechReport) 等学术出版物的信息, 如题目, 作者以及出版年份等。除此之外, 系统还存储了不同类型出版物的一些特有信息:

(1) 对于会议文章, 系统还记录了会议名称, 召开时间以及召开地点;

(2) 对于期刊文章, 系统还记录了期刊名称, 出版月份, 期号以及主办单位;

(3) 对于校内技术报告, 系统还记录了由学校分配的唯一 ID。

3. 查询制定会议集 (Proceedings) 或某个期刊特定期 (Edition) 的所有文章。会议集包含了发表在该会议 (在某个特定时间段, 特定地点召开) 上的所有文章。期刊的每一期在特定时间发行, 其中包含若干篇文章。

4. 下载出版物。系统记录每个出版物被下载的次数。

5. 查询引用了某篇出版物的所有出版物。在学术出版物中引用他人或早期的文献作为相关工作或背景资料是很常见的现象。用户也可以在系统中为某篇出版物注册引用通知, 若有新的出版物引用该出版物, 系统将发送电子邮件通知该用户。

现在采用面向对象方法对该系统进行开发, 得到系统的初始设计类图如图 3-1 所示。

【问题 1】(9 分)

根据说明中的描述, 给出图 3-1 中 C1~C9 所对应的类名。

【问题 2】(4 分)

根据说明中的描述, 给出图 3-1 中类 C6~C9 的属性。

【问题 3】(2 分)

图 3-1 中包含了那种设计模式? 实现的是该系统的哪个功能?

试题四 (共 15 分)

阅读下列说明和 C 代码, 回答问题 1 至问题 2, 将解答写在答题纸的对应栏内

【说明】

一个无向连通图 G 上的哈密顿 (Hamilton) 回路是指从图 G 上的某个顶点出发, 经过图上所有其他顶点一次且仅一次, 最后回到该顶点的路径。一种求解无向图上的哈密顿回路算法的基本思想如下:

假设图 G 存在一个从顶点 u_0 出发的哈密顿回路 $u_0-u_1-u_2-u_3-\dots-u_0-u_{n-1}-u_0$ 。算法从顶点 u_0 出发, 访问该顶点的一个未被访问的邻接顶点 u_1 , 接着从顶点 u_1 出发, 访问 u_1 的一个未被访问的邻接顶点 u_2 , ...。对顶点 u_i , 重复进行以下操作: 访问 u_i 的一个未被访问的邻接顶点 u_{i+1} ; 若 u_i 的所有邻接顶点均已被访问, 则返回到顶点 u_{i-1} , 考虑 u_{i-1} 的下一个未被访问的邻接顶点, 仍记为 u_i ; 直到找到一个哈密顿回路或者找不到哈密顿回路, 算法结束。

【C 代码】

下面是算法的 C 语言实现。

(1) 常量和变量说明

n: 图 G 中的顶点数

c[]: 图 G 的邻接矩阵

k: 统计变量, 当前已经访问的顶点数为 k+1

x[k]: 第 k 个访问的顶点编号, 从 0 开始

visited[x[k]]: 第 k 个顶点的访问标志, 0 表示未访问, 1 表示已访问

(2) C 程序

```
#include<stdio.h>
#include<stdlib.h>
#define MAX 4

Void Hamilton(int n,int x[MAX],int c[MAX][MAX]){
    int i;
    int visited[MAX];
    int k;
    /*初始化 x 数组和 visited 数组*/
    for(i=0;i<n;i++){
        x[i]=0;
        Visited[i]=0;
    }
    /*访问起初顶点*/
    K=0;
    (1) ;
    x[0]=0;
    k=k+1;
    /*访问其它顶点*/
    while(k>0){
        x[k]=x[k]+1;
        while(x[k]<n){
            if((2) && c[x[k-1]][x[k]]==1){/*领接顶点 x[k]未被访问过*/
                break;
            }
            else{
                x[k]=x[k]+1;
            }
        }
        if(x[k]<n&& k==n-1&& (3) ){/*找到一条哈密尔顿回路*/
            for(k=0;k<n;k++){
                printf("%d--",x[k]);/*输出哈密尔顿回路*/
            }
            printf("%d\n",x[0]);
            return;
        }
        else if(x[k]&& k<n-1){/*设置当前顶点的访问标志, 继续下一个顶点*/
            (4) ;
            k=k+1;
        }
    }
}
```

```
else { /*没有未被访问过的邻接顶点，回退到上一个顶点*/
    x[k]=0;
    visited[x[k]]=0;
    ____ (5) ____;
}
}
```

【问题 1】(10 分)

根据题干说明，填充 C 代码中的空 (1) ~ (5)。

【问题 2】(5 分)

根据题干说明和 C 代码，算法采用的设计策略是 (6)，该方法在遍历图的顶点时，采用的是 (7) 方法 (深度优先或广度优先)。

试题五 (共 15 分)

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某图像预览程序要求能够查看 BMP, JPEG 和 GIF 三种格式的文件，且能够在 Windows 和 Linux 两种操作系统上运行。程序需具有较好的扩展性以支持新的文件格式和操作系统。为满足上述需求并减少所需生成的子类数目，现采用桥接 (Bridge) 模式进行设计，得到如图 5.1 所示的类图。

【c++代码】

```
#include<iostream>
#include<string>
Using namespace std;

class matrix { //各种格式的文件最终都被转化为像素矩阵
    //此处代码省略
};
class Implement {
Public:
    ____ (1) ____; //显示像素矩阵 m
};
class WinImp: public Implementor {
Public:
    Void doPaint(Matrix m) { /*调用 Windows 系统的绘制函数绘制像素矩阵*/ }
};
class LinuxImp: public Implementor {
public:
    Void doPaint(Matrix m) { /*调用 Linux 系统的绘制函数绘制像素矩阵*/ }
};
class Imag {
public:
    void setImp(Implementor *imp) { this.imp=imp; }
    virtual void parseFile(String fileName)=0;
};
```

```
protected:
    Implenor *imp;
};
class BMPImage:public Image{
    //此处代码省略
};
class GIFImage:public Image{
public:
    void parseFile(String fileName){
        //此处解析 GIF 文件并获取一个像素矩阵对象 m
        __ (2) __; //显示像素矩阵 m
    }
};
class JPEGImage:public Image{
    //此处代码省略
};
int main(){
    public static void main(String[] args){
        //在 Linux 操作系统上查看 demo.gif 图像文件
        Image img=__ (3) __;
        Implenor imgImp=__ (4) __;
        __ (5) __;
        img.parseFile("demo.gif");
    }
}
```

试题六共 15 分)

阅读下列说明和 Java 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某图像预览程序要求能够查看 BMP，JPEG 和 GIF 三种格式的文件，且能够在 Windows 和 Linux 两种操作系统上运行。程序需具有较好的扩展性以支持新的文件格式和操作系统。为满足上述需求并减少所需生成的子类数目，现采用桥接（Bridge）模式进行设计，得到如图 5.1 所示的类图。

【Java 代码】

```
import java.util.*;
class Matrix{//各种格式的文件最终都被转化为像素矩阵
    //此处代码省略
};
abstract class Implement{
    public __ (1) __; //显示像素矩阵 m
};
class WinImp:public Implementor{
    public void doPaint(Matrix m){/*调用 Windows 系统的绘制函数绘制像素矩阵*/}
};
```

```
class LinuxImp: public Implementor{
    public Void doPaint(Matrix m){/*调用 Linux 系统的绘制函数绘制像素矩阵*/}
};
class Image{
    public void setImp(Implementor *imp){this.imp=imp;}
    public virtual void parseFile(String fileName)=0;
    protected   Implementor *imp;
};
class BMPImage:public Image{
    //此处代码省略
};
class GIFImage:public Image{
    public Void parseFile(String fileName){
        //此处解析 GIF 文件并获取一个像素矩阵对象 m
        ____ (2) ____; //显示像素矩阵 m
    }
};
class JPEGImage:public Image{
    //此处代码省略
};
class main(){
    public static void main(String[] args){
        //在 Linux 操作系统上查看 demo.gif 图像文件
        Image image=____ (3) ____;
        Implementor imageImp=____ (4) ____;
        ____ (5) ____;
        image.parseFile("demo.gif");
    }
}
```

试题答案与解析

试题一:

【问题一】E1: 用户; E2: 商家; E3: 单车

【问题二】D1: 用户信息文件; D2: 单车信息文件; D3: 行程信息文件;

D4: 计费规则信息文件; D5: 单车故障信息文件

【问题三】

起点	终点	名称
P3	E1	开锁密码
P3	E1	行程/费用
P3	D2	单车状态
P3	E3	开锁指令
D3	P3	行程信息 或 使用时间
D2	P3	计费规则
D3	P7	行程信息
P4	D2	单车状态

【问题四】扫码/手动开锁，骑行单车，锁车结账

【试题分析】

本题考查面向结构化软件开发方法中需求分析阶段使用的数据流图（DFD图）。作答时，建议先看问题，划出关键词，然后边阅读文字描述边作答，每阅读一句都需仔细分析是否存在对应的数据流，检查相应的数据流图是否缺少相应的数据流。

【问题一】需要填写外部实体，外部实体为不属于软件本身但是又与当前软件有交互关系的外部的人，软件，硬件，组织结构，数据库系统等。在做的是需仔细的对每一个阅读到的外部实体（一般为名词）高度重视。

【问题二】考察数据存储文件，还需要对阅读到的“...文件”或“...表”等能够存储数据的媒介词汇高度重视。

【问题三】不仅仅通过阅读文字描述来作答，同时也要使用父图与子图的数据守恒原则进行作答。

根据描述“用户在 app 端输入手机号并获取验证码后进行注册，将用户信息进行存储”并对照图 1-2 中 P1 加工和 E1 实体处可知 E1 为实体“用户”，D1 为数据存储文件“用户信息文件”。根据描述“...通过扫描二维码或手动输入编码获取开锁密码，系统发送开锁指令进行开锁，系统修改单车状态，新建单车行程...”并对照图 1-2 的加工 P3 处可知缺少一条从 P3 至实体 E3 的数据流“开锁指令”，且缺少一条从 P3 至 D2 的数据流“单车状态”；根据 P4 流入 D2 的数据流“单车基础信息”容易知道 D2 为“单车信息文件”；根据 P3 流入 D3 的数据流名称“单车行程/费用”可知 D3 为“行程信息文件”；根据描述“用户停止使用或手动锁车并结束行程后，系统根据已设置好的计费规则及使用时间自动结算，更新本次骑行的费用并显示给用户，用户确认支付后，记录行程的支付状态。系统还将重置单车的开锁密码和单车状态。”并对比 P3 加工处可知缺少一条由 D3 流向加工 P3 的数据流“计费规则”和 D3 流向 P4 的数据流“使用时间”以便 P3 计算行程费用，同时缺少一条由 P3 流向实体 E1 的数据流“行程及费用”。

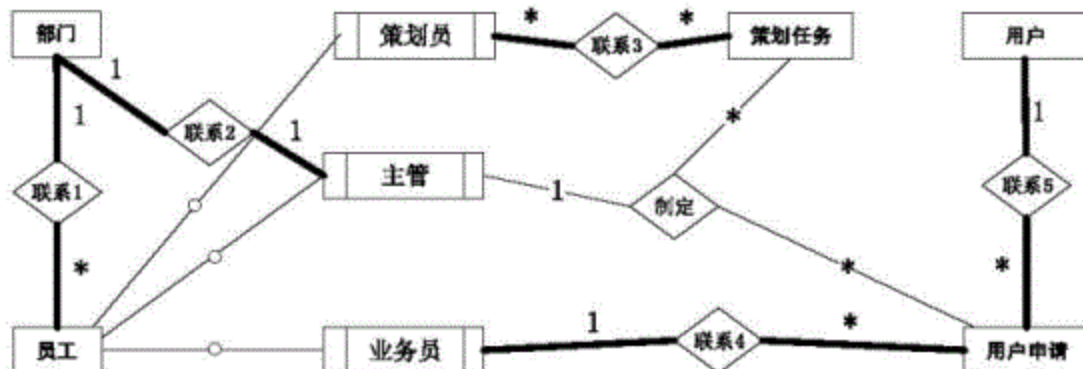
根据描述“①查询。用户可以查看行程列表和行程详细信息。”并对比加工 P4 处可知 D5 为“单车故障信息文件”；根据描述“...商家对单车基础信息，状态等进行管理，对计费规则进行设置并存储。”并对比加工 P4 周边处可知 E2 为“商家”，且缺少一条从 P4 流向 D2 的数据流“状态信息”；根据“单车监控。对单车，故障，行程等进行查询统计。”值缺少一条由 D3 流向加工 P7 的数据流“行程信息”。

最后根据图 1-1 以及图 1-2 的对比，即子图和父图数据守恒原则，知图 1-2 中还缺少一条由加工 P3 流向 E1 的数据流“开锁密码”。

根据“2）使用单车”下方的描述，使用单车可以分解为“扫码/手动开锁，骑行单车，锁车结账”三个子加工。

试题二：

【问题一】



其中粗线部分是答案。

【问题二】(a) 部门号, 职位 (b) 用户号, 银行账号 (c) 预算费用, 业务员 (d) 要求完成时间, 主管

【问题三】

“用户申请”关系模式主键: 申请号, 外键: 申请号, 业务员, 用户号;

“策划任务”关系模式主键: 申请号, 外键: 主管, 申请号

【问题四】

“执行”关系模式的主键为全码是错误的, 因为“申请号”与“策划员”的组合 (申请号, 策划员) 即使唯一确定执行关系中的一个元组数据。

【试题解析】

此类题先阅读问题, 画出关键字, 再一边仔细阅读文字描述, 一边看图, 一边看关系模式一边作答。

根据文字描述“每个部门只有一名主管, 只负责本部门的工作, 且主管参照员工关系的员工号”可知图 2-1 (后统称 E-R 图) 中实体“部门”与“主管”之间应补充 1:1 的联系; 根据“一个部门有多名员工, 每名员工属于且仅属于一个部门”可知 E-R 中实体“部门”和“员工”之间缺少 1: * 的联系, 且关系模式“员工”中空 (a) 处填写“部门号”字段作为外键以实现两表的参照完整性。根据描述“员工信息包括员工号, 姓名, 职位, 联系方式和薪资。”可知 (a) 处还缺“职位”字段。根据“一名业务员可以受理多名用户申请, 但一个用户申请只能由一个业务员受理。”可知 E-R 图中“业务员”与“用户申请”之间缺少 1: * 的联系, 且应将“1”端 (业务端) 的主键 (业务员) 加入到“*”端 (用户申请端) 中, 为了方便理解, 加入的字段为“业务员”作为外键使用, 故空 (c) 处应包括“业务员”。根据“用户信息包括用户号, 用户名, 银行账号, 电话, 联系地址。用户号唯一标识用户信息中的每一个元组。”可知 (b) 处应填“用户号”和“银行账号”, 且“用户号”是主键。根据“用户申请信息包括申请号, 用户号, 会议日期, 天数, 参会人数, 地点, 预算费用和受理标志。申请号唯一标识用户申请信息中的每一个元组, 且一个用户可以提供多个申请, 但一个用户申请只对应一个用户号。”可知 E-R 图中“用户”与“用户申请”之间缺 1: * 的联系, 且空 (c) 处为“预算费用”, 该表主键为“申请号”。根据“申请号”。根据“策划任务包括申请号, 任务明显和要求完成时间。申请号唯一标识策划任务的每一个元组。”可知“申请号”为“策划任务”的主键。根据“一个策划任务只对应一个已受理的用户申请, 但一个策划任务可由多名策划员参与执行, 且一名策划员可以参与执行多项策划任务。”可知 E-R 图中的“策划员”与“策划任务”之间缺少 *: * 的联系, 此联系其实就对应关系模式“执行”。

在作答时, 要注意概念模型 (E-R 图) 与逻辑模型 (关系模式) 的对应关系, 在 E-R 图中的部门, 员工, 策划任务, 用户, 用户申请, 策划员与策划任务之间的联系都有对应的

关系模式（E-R 图中的子实体就对应父实体的关系模式），而联系“制定”未转换为关系模式，那么主管与策划任务之间的参照关系需要将主管（“1”端）的主键“员工号”加入到策划任务（*端）中作为外键，为了方便识别，更名为“主管编号”或“主管”。由于主管已经与策划任务之间建立了参照关系，而策划任务与用户申请又是 1 对 1 的联系，故主管与用户申请之间的参照关系可通过主管与策划任务之间的参照关系间接体现，故用户申请中无须加入主管的主键字段。

“执行”关系模式的主键为全码是错误的，因为“申请号”与“策划员”的组合即能唯一确定关系中的一个元组数据。

试题三

【问题一】C1：用户；C2：系统用户或 User；C3：学生或 Student；C4：教师或 Factual；C5：其他工作人员或 Staff；C6：出版物或 Publication；C7：会议文章或 ConfPaper；C8：期刊文章或 JournalArticle；C9：校内技术报告或 TechReport（注意：C3，C4，C5 可交换）

【问题二】C6 的属性：题目，作者，出版年份，下载次数；C7：会议名称，召开时间，召开地点；C8 的属性：期刊名称，出版月份，期号，主办单位；C9 的属性：ID

【问题三】使用了观察者设计模式（又称“发布-订阅”模式），定义了一种一对多的依赖关系，在题中，某出版物是观察者，当被观察者（引用某出版物的其他出版物）出现时，则出版物会收到其被引用的通知，从而系统发送邮件给相应的作者。

【试题解析】根据描述“系统的用户（User）仅限于该大学的学生（Student），教师（Faculty）和其他工作人员（Staff）。”可知用户（User）应为父类型，而学生，教师，其他工作人员都是子类型，它们之间是一种“is-a”的泛化关系，这四个类可对应到类图中 C2 为父类，C3，C4 以及 C5 为子类处，C2 为“系统用户”，C3，C4，C5 依次“学生”，“教师”，“其他工作人员”。根据描述“查询某个作者（Author）的所有出版物。系统中保存了会议文章（ConfPaper），期刊文章（JournalArticle）和校内技术报告（TechReport）等学术出版物的信息”可知“会议文章”，“校内技术报告”都是“出版物”的子类型，对应到类图中，C6 应为“出版物”，C7 与会议集（Proceedings）有聚合关系，故 C7 为“会议文章”，同理 C8 应为“期刊文章”，C9 为“校内技术报告”。纵观整个类图，C1 为 C2（系统用户（User））和 Author 的父类型，故 C1 填写“用户”，其中包括了学生，教师，其它工作人员，作者的共同属性如登录信息等。

根据描述“查询某位作者（Author）的所有出版物...等学术出版物的信息，如题目，作者以及出版年份等。”及“下载出版物。系统记录每个出版物被下载的次数。”可知 C6 中应包含属性“题目”，“作者”，“出版年份”，“下载次数”，这些信息都是每个派生类型所共用的，故抽象到共同的父类型中，派生类继承使用即可；派生类 C7，C8 以 C9 除了拥有从父类型继承下来的属性外，还拥有自己特定的属性。根据题目文字描述 C7 应该定义的特殊属性为“会议名称”，“召开时间”，“召开地点”，C8 应该自己定义的特殊属性为“期刊名称”，“出版月份”，“期号”，“主办单位”，C9 的是“ID”。

使用了观察者设计模式，定义了一种一对多的依赖关系，让多个观察者对象同时监听某个主题对象。这个主题对象在状态发生变化时，会通知所有观察者对象，是它们能够自动更新自己。在本题中，某出版物是观察者，当被观察者（引用某出版物的其他出版物）出现时，则出版物会收到其被引用的通知，从而系统发送邮件给相应的作者。

试题四

(1) visited[0]=1 (2) visited[x[k]]==0 (3) c[x[k]][0]==1 (4) visited[x[k]]=1

(5) $k=k-1$ 或 $k--$ 或 $-k$ (6) 回溯法 (7) 深度优先

试题解析:

问题(1)处及上下几行代码(while 循环之前)是默认从 0 号顶点开始,“ $x=[0]=0$ ”表示 0 号顶点被访问过了,“ $k=k+1$ ”也表示已经找到一个满足条件的顶点,故空(1)处肯定是设置 0 号顶点已经被访问过了,应该填“ $visited[0]=1$ ”。

空(2)处根据注释知邻接顶点 $x[k]$ 未被访问过则执行 break, 则 $x[k]$ 号顶点未被访问成立的判断条件是“ $visited[x[k]]=0$ ”, 即(2)的答案。“ $c[x[k-1]][x[k]]==1$ ”是判断之前已经被访问过的顶点($x[k-1]$)与 $x[k]$ 是否为相邻顶点。

空(3)处的 if 判断表达式“找到一条哈密尔顿回路”, 成立条件为 $x[k]<n$, 且 $k==n-1$, 同时还要满足第 $x[k]$ 顶点为被访问过(空(2)处已经判断), 最后还要保证 $x[k]$ 号顶点与 0 号顶点之间有边(判断条件 $c[x[k]][0]==1$)才行, 故空(3)处应该填写“ $c[x[k]][0]==1$ ”。

空(4)处为“设置当前顶点的访问标志, 继续下一个顶点”, 则 k 应该加 1, 且应该设置 $x[k]$ 号顶点被访问过, 即空(4)应该填写“ $visited[x[k]]=1$ ”。

空(5)处所属的 else 代码块表示“没有未被访问过的邻接顶点, 回退到上一个顶点”, 则应该进行回溯, 回退到上一个顶点, 回溯的过程即使取消前一步因为“试探”而做的操作, 即取消之前“试探”过程中设置的顶点编号($x[k]=0$), 取消之前“试探”过程中访问过的顶点($visited[x[k]]=0$), 取消之前因为“试探”而增加的顶点数量($k=k-1$), 故空(5)应该填写“ $k=k-1$ ”(或 $k--$ 或 $-k$)。

算法中, 如下的代码块即使去查找与 $x[k-1]$ 号顶点相邻的顶点(从 $x[k]$ 号开始“试探”), 找到一个马上执行关键字 break (即结束循环), 然后执行该 while 循环后的代码块, 之后的过程将不再查找 $x[k-1]$ 号顶点的其他相邻顶点, 如果 $x[k]$ 号顶点不满足条件, 则执行循环中 else 部分代码, 即继续“试探” $x[k]+1$ 号顶点。如果在找到一个相邻顶点的情况下, 还有继续去搜索其他的相邻顶点, 则为广度优先方式, 本题显然不是, 而是深度优先。

```
while(x[k]<n){
    if(____(2)____ && c[x[k-1]][x[k]]==1){/*邻接顶点 x[k] 未被访问过*/
        break;
    }
    else{
        x[k]=x[k]+1;
    }
}
```

根据以上分析, 在结合以下的代码块, 次代码的功能为回退到上一个顶点继续搜索上一个顶点的其它相邻顶点, 同时在回溯的过程中要取消之前因为“试探”而进行的操作。

```
else {/*没有未被访问过的邻接顶点, 回退到上一个顶点*/
    x[k]=0;
    visited[x[k]]=0;
    ____ (5) ____;
}
```

通过以上分析, 本题使用的是回溯法, 用它可以系统地搜索一个问题的所有解或任一解。回溯法是一个既有系统性又带有跳跃性的搜索算法。它在包含问题所有解的解空间树中, 按照深度优先的策略, 从根结点出发搜索空间树, 算法搜索值解空间树的任一节点时, 总是先判断该点是否肯定不包含问题的解。如果肯定不包含, 则跳过以该节点为根的子树的系统, 逐层向其祖先节点回溯, 否则, 进入该子树, 继续按深度优先的策略进行搜

索。只要搜索到任一解就可以结束了。

试题五

(1) virtual void doPaint(Matrix m)=0; (2) imp->doPaint(m) (3) new
GIFImage()
(4) new LinuxImp() (5) imp->setImp(imageImp)

试题六

(1) abstract void doPaint(Matrix m) (2) imag.doPaint(m) (3) new GIFImage()
(4) new LinuxImp() (5) image.setImp(imageImp)